in28minutes.com · Interview Preparation Series

GIT

Interview Guide

4 Chapters · 35 Questions & Answers · June 2026

Table of Contents

CHAPTER 1

Introduction to Git

10 Questions & Answers

1 Introduction to Git

Q1 Why is Version Control Essential?

What is Version Control?

- Version Control Systems (VCS): Tools that track changes to files over time, allowing developers to manage code efficiently and collaborate without conflicts
- Purpose: Ensures that code history is maintained, and developers can work together without overwriting each others work

Why is Version Control Essential?

- Collaboration: Multiple developers can work on the same project without overwriting each others work
- History Tracking: Easily view, compare, and restore previous versions of code
- Branching: Developers can experiment with new features in separate branches without affecting the main code base

Git / Shell

```
$ git log --oneline app.py
9e7c2f5 (Ethan) Add logging to greet()
a3d1cbe (Deepa) Fix typo in greeting
6b2e9a1 (Charlie) Refactor: move greeting to main()
f1a72bb (Bob) Add greeting function
d0a1b22 (Alice) Initial version of app.py
```

Q2 How Does Git Distributed Version Control Improve Collaboration and Efficiency?

Types of VCS:

- Centralized Version Control Systems (CVCS): A single central server stores all versions of the code (e.g., SVN, CVS)
- Distributed Version Control Systems (DVCS): Every developer has a complete copy of the repository (e.g., Git, Mercurial)

Comparison

Aspect	Centralized VCS (CVCS)
Repository Location	Single central server
Offline Work (Check History,..)	Limited
Single Point of Failure	Yes (central server outage)
Backup	Central server must be backed up
Performance	Network-dependent

How Does Git's Distributed Version Control Improve Collaboration and Efficiency?

- Complete History for Everyone: Every developer's local copy includes the full project history, making offline work possible
- Includes All Branches: Local clones contain all remote branches that were fetched, allowing developers to switch, create, or merge branches offline
- Faster Operations: Most Git commands (like commit, diff) are local, providing fast performance
- No Single Point of Failure: If the main server goes down, any local copy can be used to restore the project

Q3 Why is Git Snapshot System a Game-Changer?

What is Snapshot-Based Systems?

- Snapshot Storage: Git captures the entire state of your project at each commit, like a photograph
- Complete Versions: Instead of storing just the changes (deltas), Git stores a complete version of changed project files
- Efficient Storage: Unchanged files are linked to previous versions rather than duplicated, saving space

How is it different from Delta-Based Systems (SVN)?

- Delta-Based Storage: Only stores the differences between file versions
- More Processing: Requires more computing to reconstruct previous versions
- Complex Branching: Changes are applied sequentially, making branching more complex

An example of Snapshot Based System

- COMMIT 1
  File Content:
  Git Storage: Stores a full snapshot of the file as-is
- COMMIT 2
  File Content:
  Git Storage: Stores a new full snapshot of the modified file
- COMMIT 3
  File Content:
  Git Storage: Stores yet another full snapshot of the file (Git optimizes behind the scenes using compression and shared objects)

An example of Delta Based System

- COMMIT 1
  File Content:
  SVN Storage: Stores the complete file for the initial version
- COMMIT 2
  Delta Stored: Change line 2 Version 2
  SVN Storage: Only the difference from version 1 is stored
- COMMIT 3
  Delta Stored: Change line 1 Hello Change line 2 Version 3
  SVN Storage: Stores only the differences (deltas) from previous version

Why is Git’s Snapshot System a Game-Changer?

- Speed: Quickly access any commit without calculating differences
- Reliability: Full file versions are always available
- Simplified Merging: Merging is faster because complete versions are available

Code

```
Welcome
Version 1
```

Code

```
Welcome
Version 1
```

Code

```
Welcome
Version 2
```

Code

```
Welcome
Version 2
```

Code

```
Hello
Version 3
```

Code

```
Hello
Version 3
```

Code

```
Welcome
Version 1
```

Code

```
Welcome
Version 1
```

Q4 Give a Short History of Git

History of Git

- The Need: In 2005, the Linux kernel development team required a powerful, distributed version control system
- The Problem: They were using a proprietary tool that was expensive, unreliable, and centralized
- The Solution: Linus Torvalds, the creator of Linux, developed Git to solve these problems
- Git is Popular Because of its Key Design Principles:
  - Speed: Git was designed to be fast, even for large projects
  - Security: Changes are securely tracked and verifiable
  - Decentralization: Developers can work independently, without relying on a central server
- 2005: Git was released as an open-source project
- 2008: GitHub launched, making Git widely accessible for collaborative projects (Git is a version control tool; GitHub is a cloud-based platform for hosting and managing Git repositories)
- 2010s: Git adoption surged as open-source communities and companies shifted away from Subversion (SVN) and other centralized tools
- 2018: Microsoft acquired GitHub, further boosting Git's presence in the enterprise market
- Today: Git is the standard for version control in software development

Key Commercial Products Built Around Git

- GitHub: Hosted Git repositories with collaboration, issue tracking, CI/CD, and security tools
- GitLab: Git repository hosting with integrated DevOps lifecycle tools (CI/CD, container registry, monitoring)
- Bitbucket (by Atlassian): Git hosting with Jira and Trello integration
- Azure Repos: Microsoft's Git repository service integrated with Azure DevOps
- AWS CodeCommit: Git-based repository service hosted by Amazon Web Services
- Cloud Source Repositories (CSR): Google Cloud's Git-compatible source control service for storing and managing code in private Git repositories

Q5 What are some of the popular Version Control Systems before Git?

Important Version Control Systems Before Git

Tool	Type	Strengths	Weaknesses
CVS (Concurrent Versions System)	Centralized (CVCS)	Simple and lightweight	Limited merge support, prone to errors in large projects
Subversion (SVN)	Centralized (CVCS)	User-friendly with basic features	Network-dependent for most operations, performance degrades with large repos or many branches
Perforce	Centralized (CVCS)	Excellent performance with large codebases and binary files (like game art and assets)	Complex setup, resource-intensive

> Comparing Git vs SVN

Feature	Git
Type	Distributed (DVCS)
Commit Model	Snapshots
Offline Work	Fully supported
Merging	Fast and efficient
Branching	Lightweight and fast
Used By	Used By Most Enterprise and Open Source Projects

Q6 What is GitHub?

- Platform on Top of Git: GitHub is a web-based platform built on top of Git that enhances what Git can do by adding collaboration and project management features
- Collaboration & Hosting Made Easy: Designed to host Git repositories and provide powerful tools for team collaboration, code review, and project tracking — think of it as Git++
- Open Source Friendly: Widely used for hosting open-source projects and connecting with the global developer community
- Public and Private Repositories: Choose between public repos for sharing with the world or private repos for secure, team-only collaboration
- User-Friendly Interface for Beginners: Offers a simple and intuitive UI for those who are new to Git, making common tasks easier to perform
- Pull Requests and Code Reviews: Enables developers to propose changes and collaborate through peer reviews before merging to main branches
- Built-In Issue Tracking: Manage bugs, feature requests, and team tasks using GitHub's issue system
- Team Discussions and Knowledge Sharing: Use the Discussions tab as a dedicated space for questions, ideas, and decisions — all in one place
- GitHub Actions for Automation: Set up workflows to automatically build, test, and deploy your code whenever changes are pushed
- GitHub Pages for Web Hosting: Host static websites straight from your repository — perfect for portfolios, documentation, or demos

Git vs Github

Feature	Git
Version Control	Distributed version control for local version management
Project Management	No built-in project management tools
CI/CD Integration	Requires manual setup of external CI/CD tools
Community	No direct community interaction

Q7 Can you give a Practical Example of Creating a New Git Repo and Pushing it to GitHub?

What is a Git Repository?

- Git repository: A Git repository stores all your code and its history — like a time machine for your project

    You need a separate git repository for each of your projects

- Tracks Every Change with Context: Git records who made each change, when they made it, and why — enabling complete change management
- Enables Safe Experimentation: Try new ideas locally without fear — if anything breaks, you can always go back to a previous version
- Local Repository for Private Development: You work on your own machine in a fully functional Git environment, even without internet (git init)
- Remote Repository for Team Collaboration: A shared repository helps your team work together on the same codebase (Create a repository on Github and git clone to your local machine)
- Simple Process for Seamless Collaboration: Work locally, commit changes, and push to the remote repo — teammates pull updates to stay in sync

Practical Example: Create a New Git Repo and Push to GitHub

The goal of this workflow is to set up version control for your project using Git and connect it to GitHub

```
Git / Shell
```

```
# STEP 1: Create a new local Git repository
git init
# WHAT: Initializes a new Git repo in current folder
# WHEN: You are starting a project from scratch

# STEP 2: Create a new file or modify files
echo "Hello Git" > hello.txt
# WHAT: Creates a file with sample content

# STEP 3: Check the status of the repo
git status
# WHAT: Shows untracked/modified/staged files
# WHY: Helps verify what will be committed

# STEP 4: Add file(s) to the staging area
git add hello.txt
# VARIATION:
# git add .    adds all modified/untracked files
# git add -A   adds all changes (incl. deleted files)

# STEP 5: Commit staged files
git commit -m "Initial commit"
# WHAT: Creates a snapshot of your staged changes
# BEST PRACTICE: Use short, clear commit messages

# STEP 6: Create a new GitHub repository
# Visit https://github.com/new
# NAME: my-first-git-repo (example)
# Keep the repo EMPTY (no README/.gitignore)

# STEP 7: Connect local repo to GitHub
git remote add origin \
https://github.com/youruser/my-first-git-repo.git
# WHAT: Adds a remote named 'origin' pointing to GitHub
# WHY: So you can push/pull code to/from GitHub

# STEP 8: Push code to GitHub
git push -u origin main
# WHAT: Pushes 'main' branch to 'origin' remote
# WHY: Publishes your local code to GitHub
# -u: Sets 'origin main' as default for future pushes
# Future pushes  just run: git push
```

Q8 How Git Organizes and Stores Your Code

- Working Directory – Where Files Live and Evolve: This is your active project folder where you create and edit files. It reflects your latest code and uncommitted changes
- Index (Staging Area) – Prepare Files for Commit: When you run git add, Git stores a snapshot of the file in the staging area, ready to be committed
- Local Object Database – Stores Full Commit History: Once you run git commit, Git creates objects like blobs, trees, and commits and stores them inside the hidden .git/objects folder. This is your full version history.
- Remote Object Database – Shared Copy in the Cloud: When you run git push, Git sends your commits to a remote repository like GitHub, GitLab, or Bitbucket so others can collaborate

Summary

- The working directory supports fast development and local testing
- The index allows intentional, controlled commits
- The local database provides complete project history and safety
- The remote repo enables collaboration, sharing, and backup

Scenarios

- Scenario 1 – Safely Try New Code Without Committing Everything You're experimenting with a new feature. The working directory lets you make and test changes freely. You can decide later what to stage and commit, keeping your version history clean.
- Scenario 2 – Commit Only What Matters You fixed a bug and added a feature in one go. Use the index (staging area) to carefully stage only the bug fix for the first commit. This improves commit quality and makes your history more meaningful.
- Scenario 3 – Roll Back to a Previous Version Something broke after a few changes. Thanks to the local object database, you can go back to a previous commit — no need to rewrite or remember anything manually.
- Scenario 4 – Sync Work with the Team After finishing your task, you push your commits. The remote object database makes those commits available to your team, so they can review and build on your work.
- Scenario 5 – Recover from Mistakes You accidentally deleted a file in the working directory. Since it was committed earlier and stored in the local object database, you can restore it easily using Git commands.
- Each layer exists to provide a clear, safe, and flexible workflow — perfect for individuals and teams alike

Q9 How a File Moves Through Git: From Untracked to Pushed

- Flow for a new file

Start as Untracked – File Not Known to Git Yet: When you create a new file, Git does not track it until you explicitly add it

Staged – Ready to Be Committed: Once added using git add, the file becomes tracked and enters Git's control flow for changes and history. File is moved into the staging area, marking it as ready for commit.

Committed – Saved in Local Clone: Use git commit to take a snapshot of staged files; they are now part of your local Git history inside the .git folder

Pushed – Synced to Remote Repository: Use git push to send your committed changes to a remote like GitHub or GitLab for backup and collaboration

Easy to Visualize Flow: File states flow like this — Untracked Tracked and Staged Committed Pushed

- Flow for an already committed file

Unmodified – No Changes Since Last Commit: The file remains in a stable state; nothing new has been changed after the last commit

Modified – You Made Changes, But Didn't Stage Yet: When you edit a file, Git marks it as modified but does not include it in the next commit unless you stage it

Committed – Change Saved in Local Clone: Use git commit to take a snapshot of staged files; they are now part of your local Git history inside the .git folder

Pushed – Synced to Remote Repository: Use git push to send your committed changes to a remote like GitHub or GitLab for backup and collaboration

Easy to Visualize Flow: File states flow like this — Modified Staged Committed Pushed

- Foundation: This lifecycle is the foundation of Git's version control — it allows developers to track, manage, and share changes confidently and consistently across teams and time

---

Q10 What happens in the background when you commit code? (git commit)

- Copy Staged Changes into the Local Object Database: Git stores the content from the staging area into a special .git/objects folder — this is your full version history
- Create a Tree Object for the Snapshot: Git builds a tree structure representing the exact state of your files at the time of the commit
- Generate a Commit Object with Metadata: Git adds information like author name, timestamp, and commit message to describe what the change was about
- Link Commit to the Snapshot (Tree): The commit object points to the tree object, creating a traceable connection between the message and the actual files
- Assign a Unique Commit ID: Every commit is given a unique ID that can be used to identify, share, or roll back to that point
- Chain Commits Together with a History Pointer: Each commit includes a reference to the previous commit, forming a chain — the full history of your project
- Why It Matters: Commits help you track progress, roll back mistakes, and understand the evolution of your code over time

> Git's power comes from using commits as the foundation for structure, traceability, and control
>
> Branches and tags point to commits — no commits, no tracking or navigation
>
> Use commits to checkout, tag, revert, or explore any version

CHAPTER 2

Git Branching

7 Questions & Answers

2 Git Branching

Q1 What is the need for Branching in Git?

What are Branches?

- Enable Parallel Work Without Interference: Branches let you work on multiple features or bug fixes independently, without affecting the main code base
- Easy to Switch Branches: Use git switch to move between branches without losing work — Git handles the transition smoothly (git checkout is the OLD alternative)
- Merge Branches to Combine Work: Once your feature is complete, you can merge it back into the main branch — keeping your work organized and your history clean
- Delete Branches After Merging: After merging, you can safely delete the feature branch to keep your repo tidy — the commits are already part of the main history
- Best Practice: Keep branches short-lived and focused — easier to manage, review, and merge
- Branch Is Just a Pointer to a Commit: A branch simply points to the latest commit on that line of development — it's lightweight and fast - no performance impact
- Automatically Moves with Every Commit: When you make a new commit, your current branch pointer moves forward to include it

Action	Command
Create a Branch	git branch <branch-name>
Switch to a Branch	git switch <branch-name>
Create and Switch	git switch -c <branch-name>
List All Branches	git branch
Delete a Branch	git branch -d <branch-name>
Force Delete	git branch -D <branch-name>

Q2 Give Examples of Different Types of Branches with a Practical Example

Key Types of Branches

- Integration Branch – Combine Work from Multiple Sources: An integration branch brings together code from different feature branches.
- Feature Branch – Focus on One Task at a Time: A feature branch is used to work on a specific feature in isolation from the rest of the project

    Create Feature Branches from Integration Branch: Start a feature branch from the latest version of the integration branch

    Merge Feature Branches into Integration Branch: Once the feature is complete and tested, merge it back into the integration branch
- Keeps the Codebase Stable and Clean: Integration branches act as control points — only reviewed and tested code is merged in

    Best Practice: Keep integration branches stable — never commit unfinished code directly to them
- Most Common Use Case: Use a dedicated feature branch for each user story, and merge it into integration branch when done.

Practical Example: Create a Feature Branch and Build a Feature

```
Git / Shell
```

```
# You are connected to the main branch

# Step 1: Create a new feature branch
git switch -c feature-homepage

# Why? Feature branches help isolate work on
# specific tasks or features. Keeps main clean.

# Step 2: Make a change in the feature branch
echo "<p>This is the homepage</p>" >> index.html

# Step 3: Stage and commit the change
git add index.html
git commit -m "Added homepage paragraph"

# Step 4: Switch back to main branch
git switch main

# Step 5: Merge feature branch into main
git merge feature-homepage

# Why? This integrates the feature into main.
# If no conflict, it's a fast-forward merge.

# Optional Step 6: Delete the feature branch
git branch -d feature-homepage

# Why? Feature is merged, so the branch is no
# longer needed. Keeps your branches clean.

# ================================================
# Version-Specific Notes:
# ================================================
# If you're using Git 2.23 or newer (recommended):
# - Use `git switch` to move between branches
# - Use `git switch -c` to create + switch

# Older versions of Git:
# - Use `git checkout` instead of `git switch`
# - Example: git checkout -b feature-homepage
# ================================================
```

Q3 Fast-Forward Merge vs Merge Commit vs Merge Conflict

- Scenario: Imagine combining two branches in a Git project — sometimes it's simple, sometimes complex
- Fast-Forward Merge – The Cleanest Path: Happens when the main branch hasn't moved ahead — Git simply moves the branch pointer forward. No extra commits. Simple and linear history.
- Merge Commit – Needed When Branches Diverge: If both branches have progressed, Git creates a new commit that brings both sets of changes together. Helps preserve a clear history of parallel work.

Two Parent Branches: Merge Commit has Two Parent Branches

Merge Conflict – Manual Attention Required: If the same lines of a file are changed in both branches, Git cannot auto-merge. You must resolve the conflict manually and complete the merge.

Resolving A Merge Confict - A Step By Step Example

- (Step 1) Create a Feature Branch from Main: Start by creating and switching to a new branch for your feature work
- (Step 2) Modify the File in the Feature Branch: Open about.txt and change the content to: Welcome to in28minutes - Learn DevOps with us! Then stage and commit the update
- (Step 3) Another Developer Updates the Same File on Main: While you're working on feature-update, another team member edits the same line in about.txt on the main branch. They change it to: Welcome to in28minutes - Learn Spring Boot! They commit their change to main and push it.
- (Step 4) You Switch Back to Main and Pull Latest Changes: Now your local main includes the other developer’s changes.
- (Step 5) You Try to Merge Your Feature Branch: Git detects that both branches changed the same line in about.txt and throws a merge conflict.
- (Step 6) Resolve the Merge Conflict Manually: Open about.txt. Git shows conflict markers like: Resolve the conflict by combining both lines or choosing one. Example: Welcome to in28minutes - Learn Spring Boot and DevOps! Save the file.
- (Step 7) Complete the Merge After Resolving:
- You Have Successfully Integrated Both Changes: This shows how real teamwork can cause merge conflicts — and how Git helps you safely resolve them.

Git / Shell

```
git switch -c feature-update
```

Git / Shell

```
git add about.txt
git commit -m "Updated about.txt in feature branch"
```

Git / Shell

```
git switch main
git pull origin main
```

Git / Shell

```
git merge feature-update
```

Code

```
<<<<<<< HEAD
Welcome to in28minutes - Learn Spring Boot!
=======
Welcome to in28minutes - Learn DevOps with us!
>>>>>>> feature-update
```

Git / Shell

```
git add about.txt
git commit -m "Resolved merge conflict in about.txt"
```

Q4 Compare Gitflow vs GitHub Flow vs Trunk-Based Development

What is Gitflow?

- Scenario: Imagine multiple developers working on features, fixes, and releases at the same time — without a clear workflow, this could easily lead to confusion and broken code. Can we bring structure to this chaos?
- Gitflow Workflow: A popular (and old) Git branching model that defines clear roles for each type of branch — making collaboration and delivery predictable and reliable
- Defines Clear Roles for Every Branch:
  - main: always contains production-ready code
  - develop: integration branch for upcoming features
  - feature/*: separate branches for individual features
  - release/*: used to prepare for a new release
- Enables Safe Parallel Development: Developers can work on their own feature branches without impacting each other or the main codebase
- Simplifies Release Management: Use a dedicated release branch to finalize, test, and polish code before going to production
- Supports Emergency Fixes Without Delay: Create a hotfix branch directly from main to fix issues quickly — then merge it into both main and develop
- Keeps Production Stable While Development Continues: Features are only merged to main after thorough testing via develop and release branches
- Ideal for Structured Teams and Long-Term Projects: Helps enforce discipline, consistency, and predictability in the software lifecycle
- Word of Caution: Gitflow introduces a lot of complexity

What is GitHub Flow?

- Scenario: Imagine you're working in a fast-moving team building a modern web app. You want to ship features quickly, collaborate with others, and keep production stable. Can we use a simple workflow?
- GitHub Flow – Simple, Modern, Effective: A lightweight Git workflow built around short-lived feature branches and pull requests
- Start from Main – Always: Every new feature or bug fix starts from the main branch
- Create a Feature Branch: Use git switch -c feature-name to isolate your work
- Commit Often, Push Regularly: Keep your branch up to date and back up your work. Small commits help trace changes and simplify debugging.
- Open a Pull Request Early: Share your work with teammates for feedback, discussion, and review. Collaboration starts even before you're done!
- Discuss, Review, Improve: Use GitHub comments, suggestions, and checks to ensure quality, catch bugs, and align with project goals
- Merge to Main After Review: Once approved and tested, merge your branch to main.
- Deploy Immediately: Merging to main means the feature is ready for production — deploy right away or let CI/CD take over
- Lightweight: Lightweight and flexible compared to Gitflow

What is Trunk Based Development?

- Scenario: Imagine a team releasing code multiple times a day. Long-lived branches would slow things down and increase merge pain. How do modern teams ship fast without chaos?
- Trunk-Based Development – One Branch to Rule Them All: A development model where everyone works in a single main branch (called "trunk") and commits frequently — keeping code always production-ready
- One Shared Branch – The Trunk: Instead of maintaining multiple long-lived branches, all changes go to a single branch (typically main or master)
- Small, Frequent Commits: Developers commit small chunks of work multiple times a day — helping catch issues early and reduce merge conflicts
- Feature Toggles for Incomplete Work: Use toggles/feature flags (conditional statements within the codebase, enabling or disabling specific features) to hide incomplete features from users while keeping code merged
- CI/CD is Mandatory: Every commit triggers tests and builds.
- Short-Lived Branches Only: If branches are used at all, they live for a few hours or a day — not weeks. Most teams commit directly to trunk after validation.
- Encourages Collective Code Ownership: Everyone works on the same codebase and takes responsibility for keeping it working
- Fast Feedback Loops: Since commits are frequent and automated testing is instant, bugs are detected early and resolved quickly

Feature	Gitflow	GitHub Flow
Release Frequency	Less frequent, planned releases	Frequent, even daily deployments
Branching Model	Uses multiple long-lived branches like main, develop, feature, release, and hotfix	Uses short-lived feature branches off main
Main Focus	Structure and process for managing features, releases, and fixes	Speed and simplicity
Ideal For	Large teams with multiple parallel efforts	Teams focused on quick iteration and delivery
Merge Conflicts	Can be delayed due to long-lived branches	Usually small due to short-lived branches
My Recommendations	Use for enterprise-level projects with formal release cycles	Use for startups or small teams that deploy often

Q5 What is a Forking?

- Scenario: Imagine you're excited to contribute to an open-source project, but you don't have permission to push changes directly to the main repository. How can you safely propose changes and collaborate?
- Forking Creates Your Own Workspace: Forking makes a personal copy of the project on your GitHub account, giving you full control to experiment and build without affecting the original
- No Need for Direct Access to the Original Repo: You don't need write permissions — perfect for external contributors and community developers
- Safe Place to Experiment and Learn: You can test, modify, and explore in your fork without worrying about breaking anything for others
- Submit Changes via Pull Requests: After making improvements in your fork, you propose changes by opening a pull request to the original repository — changes are reviewed before being merged
- Keeps the Original Project Protected: All changes are reviewed and approved before merging — ensures quality, security, and project integrity
- Easy to Sync With Latest Updates: Even though you're working in your fork, you can regularly fetch changes from the original repo to stay up to date
- Empowers Open-Source Collaboration at Scale: Forking allows thousands of developers to work in parallel without stepping on each other

How does the Forking Workflow look like?

- (Step 1) Fork the Repository on GitHub: Visit the original repository on GitHub and click the "Fork" button to create your own personal copy under your GitHub account Why? This gives you full access to make changes safely without affecting the original project
- (Step 2) Clone Your Fork Locally: Why? This brings your forked copy to your local machine so you can start working on it
- (Step 3) Add the Original Repo as an Upstream Remote: Why? This lets you fetch updates from the original project in case new changes are made there
- (Step 4) Create a Branch for Your Work: Why? Working in a separate branch keeps your changes organized and makes collaboration easier
- (Step 5) Make Changes and Commit: Edit files, add new features or fix bugs. Then stage and commit your changes Why? Commits save your progress with context — what changed and why
- (Step 6) Push Your Branch to Your Fork: Why? This uploads your branch to your GitHub fork so others can view and review it
- (Step 7) Open a Pull Request: Go to your fork on GitHub, switch to your branch, and click "Compare & pull request" Why? This is how you propose your changes to the original project maintainers — they can review, discuss, and merge if approved

```
Git / Shell
```

```
git clone https://github.com/your-username/project-name.git
```

Git / Shell

```
git remote add upstream \
https://github.com/original-owner/project-name.git
```

Git / Shell

```
git switch -c feature-improvement
```

Git / Shell

```
git add .
git commit -m "Added feature improvement"
```

Git / Shell

```
git push origin feature-improvement
```

Q6 Forking vs Cloning

- Scenario: Imagine two different needs — one where you want to contribute to a project you don’t own, and another where you simply want to use or explore the code. Should you fork or just clone?
- Forking Gives You a Personal Copy on GitHub: Forking creates a separate repository under your GitHub account. You own it. You can push changes, create branches, and open pull requests to the original project.

    Cloning Brings Code to Your Local Machine: Cloning simply copies the repository to your local system. You can explore, edit, and test code — but you don’t get a hosted copy on GitHub unless you manually create one.
- Fork When You Want to Contribute Publicly: If you want to contribute to an open-source project without having write access, fork it first, then clone your fork.

    Clone When You Just Want to Use or Learn: If you're just studying the code or using it as a dependency in your own project, cloning is enough.
- Forks Are Tracked on GitHub: GitHub knows a fork’s relationship with the original repository, which helps in tracking contributions and simplifying pull requests.

    Clones Are Not Connected by Default: A clone has no GitHub-level relationship with the original project. You’d have to manually add remotes if you want to contribute
- Forking Enables Pull Requests Easily: After committing changes to your fork, you can propose them using GitHub’s Pull Request feature.

    Cloning Requires Manual Setup for Contributions: If you clone directly from the original repo, you cannot push unless you have permissions — contributing requires more setup.

Action	Use Forking When...
Learning	You don't need to fork
Contributing to a Project	You want to contribute to a project you don’t own
Remote Management	You need a cloud-hosted editable copy

Q7 What is Detached HEAD in Git?

- Detached HEAD – Not Pointing to a Branch: When you run git checkout <commit-id>, Git moves HEAD to a specific commit instead of a branch — you're now in a detached state
- You Are Not on a Branch Anymore: In this mode, HEAD is not following any branch, so new commits won't belong to a named branch unless you explicitly create one
- Risk of Losing Work – Commits May Be Abandoned: If you make commits in detached HEAD mode and switch away without saving them, those commits can be lost
- Safe Usage – Read-Only or Temporary Exploration: This mode is useful when you want to explore an older commit, test a bug, or browse history without making changes
- Want to Keep Your Work? Create a Branch: If you do commit something and want to keep it, run git switch -c <branch-name> to save your changes to a new branch
- Why This Matters: Understanding detached HEAD helps avoid accidentally losing work or making changes you can’t trace
- Most Common Use Case: Inspecting an older state of the project without affecting any active branches
- Pro Tip: Always create a branch before committing if you plan to keep the changes
- Important to Remember: Detached HEAD is not dangerous, but it requires care — you're off the main road, so mark your trail if you plan to return to it later!

CHAPTER 3

Git - Other Features You Need To Know

9 Questions & Answers

3 Git - Other Features You Need To Know

Q1 How Can You Move a Specific Commit to Another Branch? (Cherry Pick)

- Cherry Pick – Apply One Commit from One Branch to Another: Use git cherry-pick <commit-id> to copy a specific commit into your current branch
- Perfect for Isolating Important Fixes: If you fixed a bug in a feature branch along with other changes, cherry-pick lets you move just the bug fix to the integration branch
- Keeps Commit History Clean: Lets you move changes without mixing unrelated code
- Watch for Conflicts: If the code has changed in both branches, cherry-pick may cause a conflict — Git will ask you to resolve it

Git / Shell

```
# View commits on other branch
git log feature-a --oneline
# SAMPLE OUTPUT:
# a1b2c3d Add footer section
# 4d5e6f7 Add subscription logic
# 8g9h0i1 Fix header layout

# Cherry Pick Specific Commit
git cherry-pick a1b2c3d
# WHAT: Applies only "Add footer section" commit to main
# WHY: You don't want the entire feature branch

# TIP: You can pick multiple commits
# git cherry-pick a1b2c3d 4d5e6f7

# If Git reports a conflict, fix it manually
# Then run:
git add .
git cherry-pick --continue
# To complete the cherry-pick process

# Push the Updated Main Branch
git push origin main

# Best Practice: Cherry-pick only tested, self-contained commits
```

Q2 How Can You Mark Important Milestones in Your Project?

- Tags – Mark Important Milestones in Your Project: Git tags are like sticky notes that highlight specific points in your commit history
- Perfect for Versioning: Use tags to mark release points like v1.0, v2.0 — making it easy to reference stable versions of your code
- Tags Don't Move: Unlike branches, tags are fixed — they always point to the same commit
- Great for References: Tags make it easy to check out old versions or compare changes between releases
- Best Practice: Always use annotated tags for official releases — they provide helpful context in the history

Git / Shell

```
# -----------------------------------
# STEP 1: Create a tag on a commit
# -----------------------------------
git tag v1.0
# WHAT: Creates a lightweight tag 'v1.0' on latest commit
# WHY: To mark important points like releases

# -----------------------------------
# STEP 2: Create an annotated tag (recommended)
# -----------------------------------
git tag -a v1.1 -m "Release version 1.1"
# WHAT: Annotated tag with message and metadata
# Includes author, date, and message

# -----------------------------------
# STEP 3: Push tag to remote
# -----------------------------------
git push origin v1.1
# WHAT: Shares the tag with collaborators

# -----------------------------------
# STEP 4: List tags
# -----------------------------------
git tag
# Lists all tags in the repository

# -----------------------------------
# STEP 5: Checkout a tag (read-only mode)
# -----------------------------------
git checkout v1.1
# WHAT: Moves to a detached HEAD at the tag

#  Tip:
# To create a branch from tag:
# git switch -c bugfix v1.1

#  Tag Use Cases:
# - Marking release versions
# - Identifying stable checkpoints
# - Used by CI/CD pipelines to trigger builds
```

Q3 How Can You Pause Your Work and Come Back Later Without Losing Anything? (Stash)

> • Stash – Temporarily Save Your Changes: Use git stash to save uncommitted changes without committing them, so you can switch branches freely
> • Clean Working Directory in Seconds: Stashing clears your changes from the working directory so you can focus on something else without losing work
> • Come Back with git stash pop: Use git stash pop to restore your saved changes and continue exactly where you left off
> • Delete When Done: Use git stash drop to delete a specific stash or git stash clear to remove all at once

```
Git / Shell
```

```
# You’re working on a new feature, but
# need to switch branches before finishing.

# ---
# STEP 1: Start with uncommitted changes
# ---
git checkout -b feature/navbar

# Make changes:
# - Edit index.html
# - Edit style.css

# Check current status
git status
# OUTPUT: Modified files (not yet committed)

# ---
# STEP 2: Save work using stash
# ---
git stash push -m "WIP: Add navbar layout"
# WHAT: Saves tracked changes with a label
# WHY: Easier to identify in stash list
# TIP: Preferred over older `git stash save` style

# VARIATION:
# git stash -u -m "WIP: Navbar with image"
# Also stashes untracked files

# ---
# STEP 3: Switch branches safely
# ---
git checkout main
# NOW: You’re free to fix something else!

# (e.g., make and commit a hotfix on main)

# ---
# STEP 4: Return to your work
# ---
git checkout feature/navbar

# List all stash entries
git stash list
# Shows:
# stash@{0}: On feature/navbar: WIP: Add navbar layout

# ---
# STEP 5: Apply the stash back
# ---
git stash apply stash@{0}
# WHAT: Applies changes, keeps stash entry
# USE WHEN: You might reuse the stash again

# VARIATION:
# git stash pop
# Applies and deletes stash entry immediately

# ---
# STEP 6: Remove stash entry manually
# ---
git stash drop stash@{0}
# OR remove all:
git stash clear
```

Q4 How Can You Keep Unwanted Files Out of Your Git Repository?

- .gitignore – Tell Git What to Ignore: The .gitignore file lets you specify which files or folders Git should not track
- Avoid Committing Unnecessary Files: Helps prevent accidental commits of files like logs, build outputs, temp files, or IDE configs
- Simple Patterns for Powerful Control: You can ignore by file name (.env), file type (*.log), or folder (/node_modules/)
- Can Be Shared Across Teams: Commit the .gitignore file to your repo so everyone follows the same ignore rules
- Best Practice: Add .gitignore early in the project — before committing files you don't want tracked

- Creating a .gitignore File:
- Common Patterns:

Code

```
touch .gitignore
```

Code

```
# Ignore specific files
secrets.txt

# Ignore file types
*.log
*.tmp

# Ignore directories
node_modules/
dist/
```

Q5 Scenario: How to Quickly Find the Commit That Introduced a Bug in Git

- Git Bisect – Find the Bug Faster: git bisect helps you locate the exact commit that introduced a bug using binary search — much faster than checking every commit manually
- (Step 1) Start the Bisect Session: Run git bisect start to begin the binary search for the buggy commit
- (Step 2) Mark the Bad Commit: Use git bisect bad <latest-commit> — this is the commit where the bug is confirmed
- (Step 3) Mark the Good Commit: Use git bisect good <oldest-working-commit> — this is where everything was working fine
- (Step 4) Git Picks a Middle Commit Automatically: Git checks out a commit halfway between good and bad
- (Step 5) Test and Report the Result: Run the app or tests.
  - If it's broken, run git bisect bad
  - If it's fine, run git bisect good
  - Git will narrow down further
- (Step 6) Repeat Until Buggy Commit Is Found: Git keeps halving the search space, you'll find the exact commit
- (Step 7) Reset After Investigation: Use git bisect reset to return to your original branch once the culprit commit is found
- Binary Search in Action: Saves massive time by testing only 5-6 commits instead of all 25

Q6 Scenario: How Can You Create Git Shortcuts to Save Time and Type Less?

- Aliases – Create Shortcuts for Git Commands: Git aliases let you define custom shortcuts for long or frequently used commands
- Save Time and Avoid Typos: Instead of typing git commit -m, you can define git cm as a quick alias
- Define Once, Use Everywhere: Set aliases using git config --global so they work in all your Git projects

```
Git / Shell
```

```
# ------------------------------------
# Git Aliases - Make Git Commands Shorter
# ------------------------------------

# STATUS
git config --global alias.st status
# git st  git status

# ADD & COMMIT
git config --global alias.cm "commit -m"
# git cm "message"  git commit -m "message"

# LOG - Pretty One-Liner
git config --global alias.lg "log --oneline --graph --all"
# git lg  visual commit history

# BRANCH
git config --global alias.br branch
# git br  git branch

# CHECKOUT
git config --global alias.co checkout
# git co main  git checkout main

# DIFFERENCE
git config --global alias.df diff
# git df  git diff

#  Add `--global` for all repos
```

Q7 How Can You Customize Git to Work the Way You Want?

- Git Config – Control How Git Behaves: git config lets you view and set Git settings that affect how Git works across projects or on your machine
- Settings Are Saved: Global Configuration is saved in ~/.gitconfig , Repo specific configuration is stored in <repo>/.git/config

```
Git / Shell
```

```
# ------------------------------------
# Global Git Configuration
# Applies to all repos for current user
# ------------------------------------

git config --global user.name "Ranga Rao"
git config --global user.email "[email protected]"
# Sets your identity for commits

git config --global init.defaultBranch main
# Sets 'main' as the default branch for new repos

git config --global core.editor "code --wait"
# Sets VS Code as default Git editor

git config --global color.ui auto
# Enables colored output for better readability

git config --global alias.st status
# Creates a shortcut: git st  git status

# ------------------------------------
# View Config Settings
# ------------------------------------

git config --list --global
# Lists all global settings

# ------------------------------------
# Local Config (Per Repo)
# ------------------------------------
# cd INTO THE REPO

git config user.name "Ranga (in28minutes)"
git config user.email "[email protected]"
# Overrides global settings for just this repo

#  Use Cases:
# - Set identity, editor, aliases
# - Customize behavior globally or locally
# - Supports team conventions and tools
```

Q8 How Do You Enforce Commit Message Standards Using Git Hooks?

- Automatic Quality Control: Git hooks act like bouncers that validate commit messages before they enter your repo
- Team Consistency: Ensures everyone follows the same standards without manual checking
- Better Documentation: Makes commit history more readable and useful for the whole team

There are a variety of hooks possible:

Hook
pre-commit
prepare-commit-msg
commit-msg
post-commit
pre-push
post-merge
post-checkout

Git / Shell

```
# ------------------------------------------
# Enforce Commit Message Standard Using a Hook
# ------------------------------------------

# STEP 1: Go to your Git repo's .git/hooks folder
cd my-repo/.git/hooks

# STEP 2: Create or modify 'commit-msg' file
touch commit-msg
chmod +x commit-msg  # Make it executable

# STEP 3: Add validation logic
# Enforces: Commit msg must start with a capital letter
# and be at least 10 characters long

nano commit-msg

# Paste the following:
#!/bin/sh

msg=$(cat "$1")

if ! echo "$msg" | grep -Eq "^[A-Z].{9,}"; then
  echo " Commit message must:"
  echo " - Start with a capital letter"
  echo " - Be at least 10 characters"
  exit 1
fi

# STEP 4: Try a bad commit and see it fail
# git commit -m "bad message"   rejected

#  Great for enforcing consistency across teams!
#  You can write complex rules using regex or shell
```

Q9 Give Examples of a Few Useful Git Commands

Command	What It Does
git log	Shows commit history for the current branch
git diff	Compares changes between areas in the repo
git restore	Restores file from index to working directory
git restore --staged	Restores file from last commit to index only
git rm	Removes file from working directory and index
git revert	Creates a commit that reverses a previous commit
git pull	Fetches from remote and merges into current branch
git blame	Shows who made each change in a file and when
git grep	Searches for text patterns in tracked files

CHAPTER 4

Git Interview - Diagrams

9 Questions & Answers

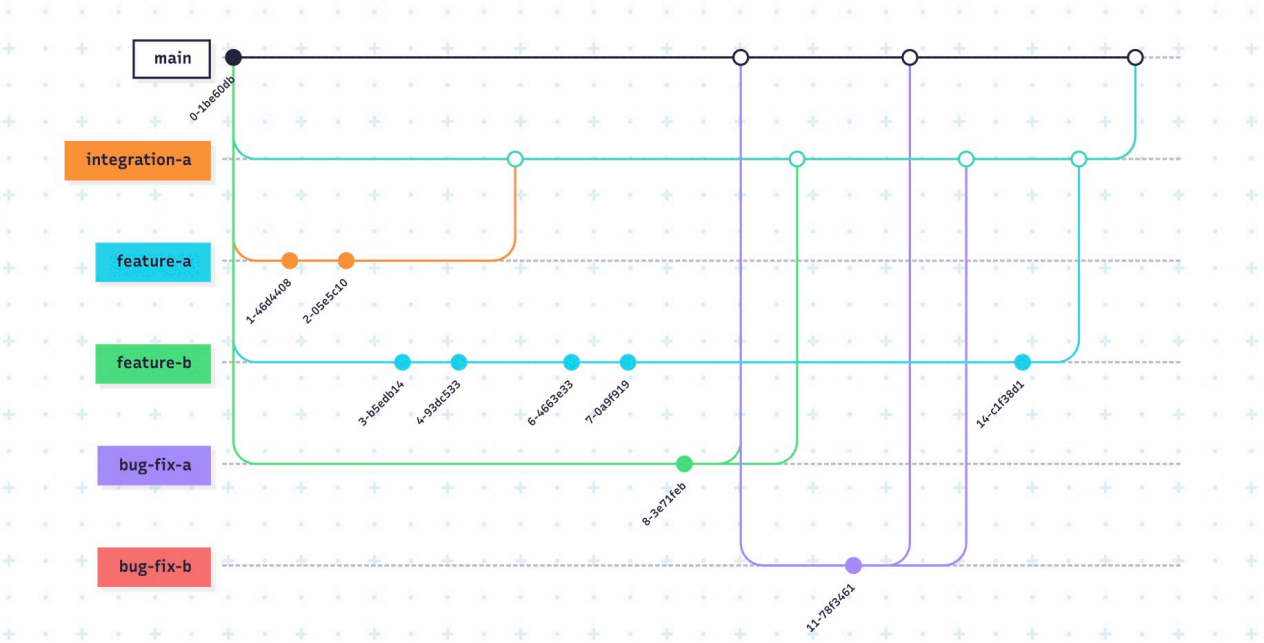
4 Git Interview - Diagrams

Q1 Branching

One Branch

Terraform State

Multiple Branches

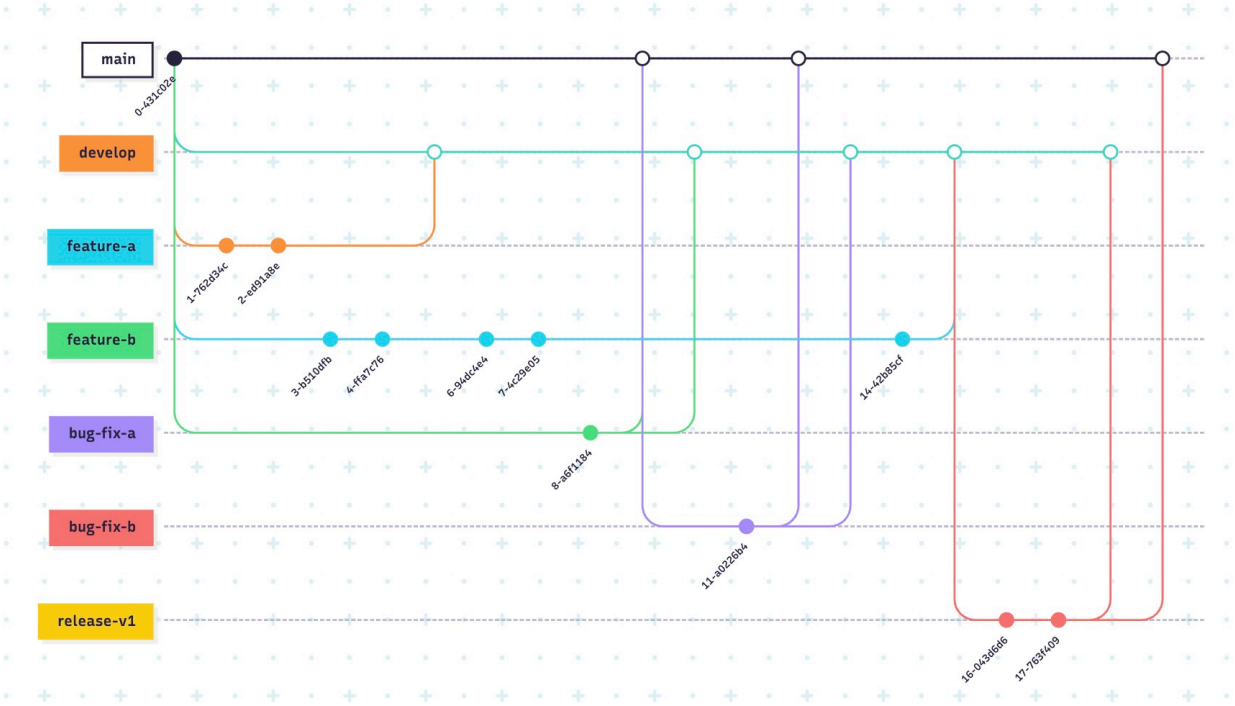


Terraform State

Gitflow

Terraform State

Q2 Git Architecture



Git / Shell

```
%% Git Lifecycle Diagram with Forward and Reverse Flows
flowchart TB
    subgraph Git Lifecycle
        WD((WorkingDirectory))
        IDX((Index/StagingArea))
        LOD((LocalObjectDatabase))
        REM((RemoteObjectDatabase))
    end

    %% Forward flow
    WD -->|git add| IDX
    IDX -->|git commit| LOD
    LOD -->|git push| REM

    %% Reverse flow
    IDX -->|git restore --staged| WD
    LOD -->|git restore / git reset| IDX
    REM -->|git fetch| LOD

    classDef nodeStyle fill:#f9f9f9,stroke:#333,stroke-width:1px;
    class WD,IDX,LOD,REM nodeStyle;
```

Q3 Git File States

Git / Shell

```
flowchart LR


    subgraph WD[Working Directory]
        UNTRACKED(Untracked<br/>NewFile)
        MODIFIED(Modified<br/>ChangedNotStaged)
        UNMODIFIED(Unmodified<br/>NoChangesSinceLastCommit)
    end

    subgraph IDX[Index / Staging Area]
        STAGED(Staged<br/>ReadyToCommit)
    end

    subgraph CLONE[Local Object Database]
        COMMITTED(CommittedToClone)
    end

    subgraph REMOTE[Remote Object Database]
        PUSHED(PushedToRemote)
    end



    %% Flows
    UNTRACKED -->|git add| STAGED
    MODIFIED -->|git add| STAGED
    STAGED -->|git commit| COMMITTED
    COMMITTED -->|git push| PUSHED
    UNMODIFIED -->|edit| MODIFIED
```

Q4 Mainline Development

Git / Shell

```
gitGraph
   commit id: "Init Repo"
   checkout main
   commit id: "Initial Commit"
   commit id: "Add Feature A"
   commit id: "Add Feature B"
   commit id: "Fix Bug"
   commit id: "Release v1.0"
```

Q5 Feature Development

Git / Shell

```
gitGraph
   commit id: "Init Repo"
   checkout main
   commit id: "Initial Commit"
   branch feature-login
   checkout feature-login
   commit id: "Add login page"
   commit id: "Add login styles"
   checkout main
   merge feature-login id: "Merge feature-login"
   branch feature-user
   checkout feature-user
   commit id: "Add user page"
   commit id: "Add user styles"
   checkout main
   merge feature-user id: "Merge feature-user"
```

Q6 Feature Development

Git / Shell

```
gitGraph
   commit id: "Init Repo"
   checkout main
   commit id: "Initial Commit"
   branch feature-login
   checkout feature-login
   commit id: "Add login page"
   commit id: "Add login styles"
   checkout main
   merge feature-login id: "Merge feature-login"
   branch feature-user
   checkout feature-user
   commit id: "Add user page"
   commit id: "Add user styles"
   checkout main
   merge feature-user id: "Merge feature-user"
```

Q7 Complex Workflow For Advanced Projects

Git / Shell

```
gitGraph
   commit
   branch integration-a
   branch feature-a
   checkout feature-a
   commit
   commit
   checkout integration-a
   branch feature-b
   checkout feature-b
   commit
   commit
   checkout integration-a
   merge feature-a
   checkout feature-b
   commit
   commit
   checkout integration-a
   checkout main
   branch bug-fix-a
   checkout bug-fix-a
   commit
   checkout main
   merge bug-fix-a
   checkout integration-a
   merge bug-fix-a
   checkout main
   branch bug-fix-b
   checkout bug-fix-b
   commit
   checkout main
   merge bug-fix-b
   checkout integration-a
   merge bug-fix-b
   checkout feature-b
   commit
   checkout integration-a
   merge feature-b
   checkout main
   merge integration-a
```

Q8 Gitflow

Git / Shell

```
gitGraph
   commit
   branch develop
   branch feature-a
   checkout feature-a
   commit
   commit
   checkout develop
   branch feature-b
   checkout feature-b
   commit
   commit
   checkout develop
   merge feature-a
   checkout feature-b
   commit
   commit
   checkout develop
   checkout main
   branch bug-fix-a
   checkout bug-fix-a
   commit
   checkout main
   merge bug-fix-a
   checkout develop
   merge bug-fix-a
   checkout main
   branch bug-fix-b
   checkout bug-fix-b
   commit
   checkout main
   merge bug-fix-b
   checkout develop
   merge bug-fix-b
   checkout feature-b
   commit
   checkout develop
   merge feature-b
   checkout develop
   branch release-v1
   commit
   commit
   checkout develop
   merge release-v1
   checkout main
   merge release-v1
```

Q9 25 commits

Git / Shell

```
gitGraph
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
   commit
```